

Study Material: Section 16.1 – Aggregate Analysis

1. Theory & In-Depth Explanation

1.1 What is Amortized Analysis?

In an **amortized analysis**, the time required to perform a sequence of data-structure operations is averaged over all the operations performed.

- **Key Guarantee:** Amortized analysis guarantees the **average performance of each operation in the worst case**.
- **Distinction from Average-Case Analysis:** Average-case analysis relies on probabilistic assumptions about the input distribution. In contrast, amortized analysis involves **no probability**; it computes an upper bound on the total time of any sequence of n operations in the worst-case scenario and divides by n .

1.2 Core Concept of Aggregate Analysis

In **aggregate analysis**, we show that for all n , a sequence of n operations takes a total worst-case time of $T(n)$.

$$\text{Amortized Cost per Operation} = \frac{T(n)}{n}$$

- **Uniform Allocation:** Aggregate analysis assigns this same average cost $\frac{T(n)}{n}$ to every operation in the sequence, even if the sequence comprises different types of operations (e.g., `PUSH`, `POP`, and `MULTIPOP`).

1.3 Case Study 1: Stack Operations with `MULTIPOP`

The Operations

1. `PUSH( $S, x$ )`: Pushes element x onto stack S . Cost = 1.
2. `POP( $S$ )`: Pops the top element of stack S . Cost = 1.
3. `MULTIPOP( $S, k$ )`: Removes the top k elements from stack S , or empties the stack if $|S| < k$.

$$\text{Actual Cost} = \min(|S|, k)$$

Naive (Loose) Worst-Case Bound

- A single MULTIPOP on a stack of size up to n takes $O(n)$ time.
- For a sequence of n operations, assuming every operation could be the most expensive operation (MULTIPOP) gives:

$$T(n) = n \times O(n) = O(n^2)$$

- While mathematically sound as an upper bound, this bound is **not tight** because a stack cannot contain more elements than have been previously pushed.

Tight Bound via Aggregate Analysis

- An object can be popped at most once for each time it is pushed onto the stack.
- In any sequence of n operations starting from an empty stack:
 - Number of PUSH operations $\leq n$.
 - Total number of objects ever pushed $\leq n$.
 - Total number of POP operations (including calls inside MULTIPOP) $\leq n$.
- Total cost $T(n) \leq \text{Cost}(\text{PUSH}) + \text{Cost}(\text{POP}) \leq n + n = 2n = O(n)$.
- **Amortized cost per operation:**

$$\frac{T(n)}{n} = \frac{O(n)}{n} = O(1)$$

1.4 Case Study 2: Incrementing a k -Bit Binary Counter

Problem Setup

- A counter counts upward from 0 using an array $A[0 \dots k-1]$ of k bits.
- Lowest-order bit: $A[0]$; Highest-order bit: $A[k-1]$.
- Counter value: $x = \sum_{i=0}^{k-1} A[i] \cdot 2^i$.
- Cost of an INCREMENT call is linear in the number of bits flipped ($0 \rightarrow 1$ or $1 \rightarrow 0$).

Naive Bound

- In the worst case, incrementing a counter with all 1s (e.g., $11 \dots 1 \rightarrow 00 \dots 0$) flips all k bits, taking $\Theta(k)$ time.
- A sequence of n operations takes $n \times O(k) = O(nk)$ time.

Aggregate Analysis via Flip Frequencies

Not all bits flip on every call:

- $A[0]$ flips every time: $\lfloor n/1 \rfloor = \lfloor n/2^0 \rfloor$ times.
- $A[1]$ flips every 2nd time: $\lfloor n/2 \rfloor = \lfloor n/2^1 \rfloor$ times.
- $A[2]$ flips every 4th time: $\lfloor n/4 \rfloor = \lfloor n/2^2 \rfloor$ times.

- In general, bit $A[i]$ flips $\lfloor n/2^i \rfloor$ times for $i = 0, 1, \dots, k-1$.

Summing the total number of bit flips across all n operations:

$$\sum_{i=0}^{k-1} \left\lfloor \frac{n}{2^i} \right\rfloor < n \sum_{i=0}^{\infty} \frac{1}{2^i} = n \left(\frac{1}{1 - 1/2} \right) = 2n$$

- **Total Cost:** $T(n) < 2n = O(n)$.
- **Amortized Cost per Operation:**

$$\frac{T(n)}{n} < \frac{2n}{n} = 2 = O(1)$$

2. Implementations & Walkthroughs

Here are the pseudocode and C++ implementations for both data structures analyzed in Section 16.1.

2.1. Augmented Stack with `MULTIPOP`

Pseudocode

```
PUSH(S, x)
1  top[S] = top[S] + 1
2  S[top[S]] = x

POP(S)
1  if STACK-EMPTY(S)
2      error "underflow"
3  else
4      top[S] = top[S] - 1
5      return S[top[S] + 1]

MULTIPOP(S, k)
1  while not STACK-EMPTY(S) and k > 0
2      POP(S)
3      k = k - 1
```

C++ Implementation

```
#include <iostream>
#include <vector>
#include <stdexcept>
```

```

#include <algorithm>

template <typename T>
class AugmentedStack {
private:
    std::vector<T> elements;

public:
    // PUSH: Cost = 1
    int push(const T& x) {
        elements.push_back(x);
        return 1; // 1 unit of cost
    }

    // POP: Cost = 1
    std::pair<T, int> pop() {
        if (elements.empty()) {
            throw std::underflow_error("Stack is empty");
        }
        T topValue = elements.back();
        elements.pop_back();
        return {topValue, 1}; // {popped_element, cost}
    }

    // MULTIPOP: Cost = min(size, k)
    std::pair<std::vector<T>, int> multipop(int k) {
        std::vector<T> popped;
        int cost = 0;

        while (!elements.empty() && k > 0) {
            popped.push_back(elements.back());
            elements.pop_back();
            k--;
            cost++;
        }
        return {popped, cost}; // {popped_elements, cost}
    }

    bool empty() const {
        return elements.empty();
    }

    size_t size() const {
        return elements.size();
    }
};

```

Walkthrough:

- `push` appends the element and records an abstract cost of 1.
 - `pop` verifies the stack is non-empty, pops the top element, and incurs a cost of 1.
 - `multipop` runs a `while` loop bounded by both stack size and k . Each successful `pop` increments the cost by 1, demonstrating that the actual cost is strictly $\min(|S|, k)$.
-

2.2. k -Bit Binary Counter

Pseudocode

```
INCREMENT(A, k)
1  i = 0
2  while i < k and A[i] == 1
3      A[i] = 0
4      i = i + 1
5  if i < k
6      A[i] = 1
```

C++ Implementation

```
#include <iostream>
#include <vector>

class BinaryCounter {
private:
    int k;
    std::vector<int> A; // A[0] is LSB, A[k-1] is MSB

public:
    explicit BinaryCounter(int bitCount) : k(bitCount), A(bitCount, 0) {}

    // INCREMENT: Returns the number of bit flips (actual cost)
    int increment() {
        int i = 0;
        int flips = 0;

        // Flip lower-order 1s to 0s (carry cascade)
        while (i < k && A[i] == 1) {
            A[i] = 0;
            flips++;
            i++;
        }

        // Flip the lowest 0 to 1
        if (i < k) {
```

```

        A[i] = 1;
        flips++;
    }

    return flips; // Total bit flips executed during this call
}

void display() const {
    for (int i = k - 1; i >= 0; i--) {
        std::cout << A[i];
    }
    std::cout << '\n';
}
};

```

Walkthrough:

- The array `A` stores bits such that `A[0]` represents the least significant bit (2^0).
- Lines 2–4 in the pseudocode (and the `while` loop in C++) reset all consecutive trailing 1s to 0, accumulating carries.
- Line 5–6 (and the `if (i < k)` condition) sets the first encountered 0 bit to 1.
- The `flips` counter tracks the exact operational cost, directly validating the aggregate analysis showing that the sum of flips across n operations remains $< 2n$.

3. Examples & Trick Questions

Trick 1 (CLRS Exercise 16.1-1): Adding MULTIPUSH

- **Question:** If we introduce a `MULTIPUSH(S, k)` operation that pushes k arbitrary elements onto the stack in $\Theta(k)$ time, does the $O(1)$ amortized bound per operation still hold?
- **Analysis:**
Consider a sequence of n operations where each operation is `MULTIPUSH(S, k)`.
 - Total items pushed = $n \cdot k$.
 - Total cost = $\Theta(nk)$.
 - Average cost per operation = $\frac{\Theta(nk)}{n} = \Theta(k)$.
If k can be as large as n , the total cost is $\Theta(n^2)$, yielding an amortized cost of $\Theta(n) \neq O(1)$.
- **Conclusion: No**, the $O(1)$ bound does not hold unless k is bounded by a constant.

Trick 2 (CLRS Exercise 16.1-2): Introducing DECREMENT

- **Question:** Show that if a DECREMENT operation is included, a sequence of n operations on an initially zero k -bit counter can cost as much as $\Theta(nk)$ time.
 - **Analysis:**
 1. Start with the counter at 0.
 2. Call INCREMENT : Value becomes 1 (00000001_2). Cost = 1.
 3. Call INCREMENT $2^{k-1} - 1$ times to reach $0111 \dots 1_2$.
 4. Now alternate between INCREMENT and DECREMENT :
 - Counter is $0111 \dots 1_2$.
 - INCREMENT $\rightarrow 1000 \dots 0_2$. All k bits flip. Cost = k .
 - DECREMENT $\rightarrow 0111 \dots 1_2$. All k bits flip. Cost = k .
 - Repeating this alternation m times costs $\Theta(m \cdot k)$.
 - **Conclusion:** A sequence of n operations alternating between these border states causes each operation to flip $\Theta(k)$ bits, resulting in a worst-case total cost of $\Theta(nk)$ and an amortized cost of $\Theta(k)$.
-

Trick 3 (CLRS Exercise 16.1-3): Operation Cost Dependent on Powers of 2

- **Question:** Suppose the i -th operation costs i if i is an exact power of 2, and 1 otherwise. Determine the amortized cost per operation using aggregate analysis for a sequence of n operations.
- **Analysis:**

Let c_i be the cost of the i -th operation:

$$c_i = \begin{cases} i & \text{if } i = 2^j \text{ for some integer } j \geq 0 \\ 1 & \text{otherwise} \end{cases}$$

Total cost $T(n) = \sum_{i=1}^n c_i$:

$$T(n) = \sum_{i=1}^n 1 + \sum_{j=0}^{\lfloor \lg n \rfloor} (2^j - 1)$$

(Note: We add 1 for every operation, then add the extra $2^j - 1$ for power-of-2 indices).
Alternatively, upper bound the powers of 2 directly:

$$T(n) \leq n + \sum_{j=0}^{\lfloor \lg n \rfloor} 2^j = n + (2^{\lfloor \lg n \rfloor + 1} - 1) < n + 2n = 3n$$

$$\text{Amortized cost} = \frac{T(n)}{n} < \frac{3n}{n} = 3 = O(1)$$

- **Conclusion:** The amortized cost per operation is $O(1)$.

4. Expected Questions & Model Answers

[2 Marks] Short-Answer Questions

Q1. Define amortized cost in the context of aggregate analysis.

Answer:

In aggregate analysis, if a sequence of n operations takes a total worst-case time of $T(n)$, the amortized cost per operation is defined as the average cost $\frac{T(n)}{n}$. This uniform cost is assigned to every operation in the sequence regardless of its individual actual runtime.

Q2. Why does amortized analysis differ fundamentally from average-case analysis?

Answer:

Average-case analysis relies on probability distributions over input instances and calculates an expected time. Amortized analysis does not use probability; it computes a guaranteed worst-case bound over any sequence of operations and averages it across the sequence.

Q3. What is the individual worst-case time of a single `MULTIPOP(S, k)` versus its amortized time in a sequence of n operations on an initially empty stack?

Answer:

- Individual worst-case time: $O(n)$ (when the stack contains up to n elements).
- Amortized time: $O(1)$ (because each element pushed can be popped at most once).

[5 Marks] Medium-Answer Questions

Q4. Use aggregate analysis to show that a sequence of n operations consisting of `PUSH`, `POP`, and `MULTIPOP` on an initially empty stack runs in $O(n)$ time.

Answer:

1. Operation Costs:

- `PUSH`: Costs 1 unit of time and adds 1 object to the stack.
- `POP`: Costs 1 unit of time and removes 1 object.
- `MULTIPOP(S, k)`: Removes $\min(|S|, k)$ objects; actual cost is linear in the number of objects popped.

2. Aggregate Bound:

- Each object placed on the stack must be introduced via a PUSH operation.
- In a sequence of n operations, at most n PUSH operations can occur, meaning at most n objects are ever pushed onto the stack.
- An object can be popped at most once. Therefore, the total number of calls to POP (including those executed inside MULTIPOP) cannot exceed the total number of PUSH operations, which is at most n .

3. Total Cost and Amortization:

$$T(n) \leq \text{Total PUSHes} + \text{Total POPs} \leq n + n = 2n = O(n)$$

Dividing the total cost $T(n)$ by the number of operations n :

$$\text{Amortized Cost} = \frac{T(n)}{n} = \frac{O(n)}{n} = O(1)$$

Thus, all three stack operations have an amortized cost of $O(1)$.

Q5. Consider a data structure where the i -th operation costs i if i is an exact power of 2, and 1 otherwise. Using aggregate analysis, prove that the amortized cost per operation is $O(1)$.

Answer:

Let c_i denote the cost of the i -th operation in a sequence of n operations:

$$c_i = \begin{cases} i & \text{if } i = 2^j \text{ for some integer } j \geq 0 \\ 1 & \text{otherwise} \end{cases}$$

Summing over all n operations to find total cost $T(n)$:

$$T(n) = \sum_{i=1}^n c_i \leq \sum_{i=1}^n 1 + \sum_{j=0}^{\lfloor \lg n \rfloor} 2^j$$

Using the geometric series formula $\sum_{j=0}^m 2^j = 2^{m+1} - 1$:

$$T(n) \leq n + \left(2^{\lfloor \lg n \rfloor + 1} - 1\right)$$

Since $2^{\lfloor \lg n \rfloor + 1} \leq 2^{\lg n + 1} = 2n$:

$$T(n) < n + 2n = 3n$$

Averaging over the n operations:

$$\text{Amortized cost per operation} = \frac{T(n)}{n} < \frac{3n}{n} = 3 = O(1)$$

Hence, the amortized cost per operation is $O(1)$.

[10 Marks] Comprehensive Analytical Question

Q6. Analyze the increment of an initially zero k -bit binary counter.

1. Explain the naive worst-case running time of a sequence of n **INCREMENT** operations.
2. Formulate and prove the tight bound using aggregate analysis.
3. Discuss what happens if a **DECREMENT** operation is also introduced.

Answer:

1. Naive Worst-Case Analysis

- In a k -bit binary counter stored in array $A[0 \dots k-1]$, the cost of **INCREMENT** is proportional to the number of bits flipped.
- In the worst case, a single **INCREMENT** flips all k bits (e.g., transition from $11 \dots 1$ to $00 \dots 0$). Thus, the worst-case cost of a single operation is $\Theta(k)$.
- For a sequence of n operations, multiplying the individual worst-case cost by n yields:

$$T(n) = n \times O(k) = O(nk)$$

- This bound is loose because not all bits flip during every **INCREMENT** call.

2. Tight Bound via Aggregate Analysis

- **Bit Flip Frequency:**
 - Bit $A[0]$ flips on every **INCREMENT** : $\lfloor n/1 \rfloor = \lfloor n/2^0 \rfloor$ times.
 - Bit $A[1]$ flips every 2nd **INCREMENT** : $\lfloor n/2 \rfloor = \lfloor n/2^1 \rfloor$ times.
 - In general, bit $A[i]$ flips every 2^i -th call, totaling $\lfloor n/2^i \rfloor$ flips for $i = 0, 1, \dots, k-1$.
 - For $i \geq k$, bits do not exist and flip 0 times.

- **Summation of Total Bit Flips:**

The total number of bit flips across n operations is:

$$T(n) = \sum_{i=0}^{k-1} \left\lfloor \frac{n}{2^i} \right\rfloor$$

Bounding the finite sum using an infinite geometric series:

$$T(n) < \sum_{i=0}^{\infty} \frac{n}{2^i} = n \sum_{i=0}^{\infty} \left(\frac{1}{2}\right)^i$$

Using the standard sum $\sum_{i=0}^{\infty} x^i = \frac{1}{1-x}$ for $|x| < 1$:

$$T(n) < n \left(\frac{1}{1 - 1/2} \right) = 2n$$

- **Amortized Cost Calculation:**

$$\text{Amortized Cost} = \frac{T(n)}{n} < \frac{2n}{n} = 2 = O(1)$$

The total time for n operations is $O(n)$, giving an amortized cost of $O(1)$ per `INCREMENT`.

3. Impact of Introducing `DECREMENT`

- Suppose we add a `DECREMENT` operation that subtracts 1 from the counter.
- Consider an adversary triggering operations that alternate around a power-of-2 boundary, such as between $2^{k-1} - 1$ and 2^{k-1} :

$$\text{Value } 2^{k-1} - 1 = 0111 \dots 1_2$$

$$\text{Value } 2^{k-1} = 1000 \dots 0_2$$

- When the counter is at $0111 \dots 1_2$:
 - Calling `INCREMENT` flips k bits (all low-order 1s become 0, and bit $k - 1$ becomes 1). Cost = k .
 - Calling `DECREMENT` immediately after flips k bits back. Cost = k .
- If a sequence of n operations alternates between `INCREMENT` and `DECREMENT` at this threshold:

$$T(n) = \sum_{j=1}^n k = \Theta(nk)$$

$$\text{Amortized Cost} = \frac{\Theta(nk)}{n} = \Theta(k)$$

- **Conclusion:** With `DECREMENT` included, the amortized cost per operation degrades from $O(1)$ to $\Theta(k)$.

Study Material: Section 16.2 – The Accounting Method

1. Theory & In-Depth Explanation

1.1 Fundamental Mechanism of the Accounting Method

In the **accounting method** of amortized analysis, different operations can be assigned different charges, called **amortized costs** ($\hat{c}_i$). Some operations are charged more than they actually cost, while others are charged less:

$$\hat{c}_i = c_i + \text{Credit Deposited/Consumed}$$

- **Credit as an Invariant:** When an operation's amortized cost $\hat{c}_i$ exceeds its actual cost c_i , the surplus ($\hat{c}_i - c_i > 0$) is stored on specific objects in the data structure as **credit**.
- **Credit Consumption:** When an expensive operation occurs whose actual cost exceeds its assigned amortized cost ($c_i > \hat{c}_i$), the deficit is paid for by consuming stored credit.

```
+-----+
| Operation i                               |
| - Amortized charge:  $\hat{c}_i$                    |
|                                           |
| If  $\hat{c}_i > c_i$ :   [Actual Cost  $c_i$ ] + [Surplus Credit ---> | Data
Structure]
| If  $\hat{c}_i < c_i$ :   [Actual Cost  $c_i$ ] <-- [Credit Consumed] -|
+-----+
```

1.2 The Credit Invariant: Equation (16.1)

To ensure that the sum of amortized costs provides an upper bound on the actual costs of any sequence of n operations, the total amortized cost must always exceed or equal the total actual cost:

$$\sum_{i=1}^n \hat{c}_i \geq \sum_{i=1}^n c_i \quad \text{for all } n \geq 1 \quad (16.1)$$

The accumulated credit stored in the data structure after n operations is:

$$\text{Credit}_n = \sum_{i=1}^n \hat{c}_i - \sum_{i=1}^n c_i$$

Critical Rule: Nonnegativity of Credit

- $\text{Credit}_n \geq 0$ must hold **at all times** for **every valid sequence** of operations.
- **Why?** If credit were ever allowed to become negative, the amortized cost accumulated up to that point would underestimate the actual work done. In that scenario, the analysis would fail to serve as a valid worst-case upper bound.

1.3 Accounting Method vs. Aggregate Analysis

Characteristic	Aggregate Analysis (Section 16.1)	Accounting Method (Section 16.2)
Cost Assignment	All operations in the sequence receive the exact same amortized cost: $T(n)/n$.	Different operation types can receive different amortized costs ($\hat{c}_i$).
Mechanism	Computes the global sum $T(n)$ directly over the whole sequence.	Localizes cost differences by associating surplus credits with physical data structure objects.
Intuition	Macro-level averaging across a timeline.	Bank account / prepayment system linked to individual elements.

1.4 Case Study 1: Stack Operations with MULTIPOP

Let \$1 represent one unit of computational cost.

- **Actual Costs (c_i):**
- PUSH: 1
- POP: 1
- MULTIPOP(S, k): $\min(|S|, k)$

Amortized Cost Assignment ($\hat{c}_i$):

- PUSH: \$2
- POP: \$0
- MULTIPOP: \$0

PUSH Action (\$2 charged):

\$1 --> Pays actual push cost

\$1 --> Deposited onto the pushed plate as credit

Stack State:

[Plate 3] (Credit: \$1)

[Plate 2] (Credit: \$1)

[Plate 1] (Credit: \$1)

POP Action (\$0 charged):

Actual cost (\$1) is paid by consuming the \$1 credit sitting on Plate 3.

MULTIPOP(S, k) (\$0 charged):

Pops $\min(|S|, k)$ plates. Each plate already carries \$1 credit, which directly covers the \$1 actual pop cost for that plate.

Verification of Invariant

- Since every plate on the stack always carries \$1 of credit, and the stack size $|S| \geq 0$:

$$\text{Total Credit} = |S| \times \$1 \geq 0$$

- The credit is never negative.
 - Total amortized cost for n operations is bounded by $2n = O(n)$. Therefore, the total actual cost is also $O(n)$.
-

1.5 Case Study 2: Incrementing a k -Bit Binary Counter

- **Actual Cost (c_i):** 1 unit of cost per bit flip ($0 \rightarrow 1$ or $1 \rightarrow 0$).

Amortized Cost Assignment:

- Setting a bit from $0 \rightarrow 1$: Amortized cost = \$2.
- \$1 pays for the actual flip ($0 \rightarrow 1$).
- \$1 is deposited onto the bit as credit.
- Resetting a bit from $1 \rightarrow 0$: Amortized cost = \$0.
- The actual cost of flipping $1 \rightarrow 0$ is paid by the \$1 credit previously deposited on that bit.

Amortized Cost of `INCREMENT` :

1. In the `while` loop, all bits that are reset from $1 \rightarrow 0$ cost \$0 in amortized terms (paid by their own stored credits).
2. At most one bit is flipped from $0 \rightarrow 1$ (line 6 of `INCREMENT`). This costs at most \$2 in amortized terms.
3. Therefore, the amortized cost $\hat{c}$ of any `INCREMENT` operation is:

$$\hat{c} \leq 2 = O(1)$$

Verification of Invariant

- At any time, every 1-bit stores \$1 credit, while 0-bits store \$0 credit.
- If the counter contains b ones, the total credit in the system is:

$$\text{Credit} = b \geq 0$$

- Since credit is always nonnegative, a sequence of n `INCREMENT` operations has total amortized cost $\leq 2n$, proving that total actual cost is $O(n)$.
-

2. Implementations & Walkthroughs

2.1 Stack with Accounting Tracking

Pseudocode

```
PUSH(S, x)
1  top[S] = top[S] + 1
2  S[top[S]] = x
3  credit[top[S]] = 1          // Store $1 credit on the newly pushed
    element
4  actual_cost = 1
5  amortized_cost = 2
6  bank_credit = bank_credit + (amortized_cost - actual_cost)

POP(S)
1  if STACK-EMPTY(S)
2      error "underflow"
3  actual_cost = 1
4  amortized_cost = 0
5  bank_credit = bank_credit - credit[top[S]] // Consume stored $1
6  top[S] = top[S] - 1
7  return S[top[S] + 1]

MULTIPOP(S, k)
1  actual_cost = 0
2  amortized_cost = 0
3  while not STACK-EMPTY(S) and k > 0
4      bank_credit = bank_credit - credit[top[S]] // Consume stored $1
5      top[S] = top[S] - 1
6      k = k - 1
7      actual_cost = actual_cost + 1
```

C++ Implementation

```
#include <iostream>
#include <vector>
#include <stdexcept>

template <typename T>
class AccountingStack {
private:
    struct Element {
        T value;
        int credit; // Holds prepayment
    };

    std::vector<Element> elements;
    long long total_actual_cost = 0;
```

```

long long total_amortized_cost = 0;

public:
    // PUSH: Actual = 1, Amortized = 2
    void push(const T& x) {
        int actual = 1;
        int amortized = 2;

        elements.push_back({x, 1}); // $1 deposited on item
        total_actual_cost += actual;
        total_amortized_cost += amortized;
    }

    // POP: Actual = 1, Amortized = 0
    T pop() {
        if (elements.empty()) {
            throw std::underflow_error("Stack is empty");
        }

        int actual = 1;
        int amortized = 0;

        // The item's $1 credit pays for the pop
        elements.pop_back();

        total_actual_cost += actual;
        total_amortized_cost += amortized;
        return elements.back().value;
    }

    // MULTIPOP: Actual = min(s, k), Amortized = 0
    void multipop(int k) {
        int actual = 0;
        int amortized = 0;

        while (!elements.empty() && k > 0) {
            elements.pop_back(); // Consumes $1 credit per item
            actual++;
            k--;
        }

        total_actual_cost += actual;
        total_amortized_cost += amortized;
    }

    long long getStoredCredit() const {
        return total_amortized_cost - total_actual_cost;
    }

    size_t size() const {

```

```

        return elements.size();
    }
};

```

Walkthrough:

- Every `push` assigns `credit = 1` to the element and records $\hat{c} = 2$.
- When `pop` or `multipop` runs, the elements are removed without additional amortized charges ($\hat{c} = 0$), because each popped item carries \$1 credit.
- `getStoredCredit()` returns `elements.size()`, which is always ≥ 0 .

2.2 *k*-Bit Counter with Credit Tracking

Pseudocode

```

INCREMENT(A, k)
1  i = 0
2  actual_cost = 0
3  while i < k and A[i] == 1
4      A[i] = 0
5      credit[i] = 0          // Consumed $1 credit on bit i
6      actual_cost = actual_cost + 1
7      i = i + 1
8  if i < k
9      A[i] = 1
10     credit[i] = 1          // Deposited $1 credit on bit i
11     actual_cost = actual_cost + 1
12     amortized_cost = 2
13 else
14     amortized_cost = 0

```

C++ Implementation

```

#include <iostream>
#include <vector>

class AccountingBinaryCounter {
private:
    int k;
    std::vector<int> A;          // Bits: 0 or 1
    std::vector<int> credit;    // Credit stored on each bit
    long long total_actual_cost = 0;
    long long total_amortized_cost = 0;

```

```

public:
    explicit AccountingBinaryCounter(int bitCount)
        : k(bitCount), A(bitCount, 0), credit(bitCount, 0) {}

    void increment() {
        int i = 0;
        int actual = 0;
        int amortized = 0;

        // Reset 1 -> 0 (paid by credit)
        while (i < k && A[i] == 1) {
            A[i] = 0;
            credit[i] = 0; // Credit consumed
            actual++;
            i++;
        }

        // Set 0 -> 1 (charged 2: 1 for flip, 1 deposited)
        if (i < k) {
            A[i] = 1;
            credit[i] = 1; // Credit deposited
            actual++;
            amortized = 2;
        }

        total_actual_cost += actual;
        total_amortized_cost += amortized;
    }

    long long getStoredCredit() const {
        return total_amortized_cost - total_actual_cost;
    }

    int countOnes() const {
        int ones = 0;
        for (int bit : A) ones += bit;
        return ones;
    }
};

```

Walkthrough:

- The `while` loop flips $1 \rightarrow 0$ and clears `credit[i]`, incurring 0 amortized charge.
- Setting bit $0 \rightarrow 1$ requires $\hat{c} = 2$, depositing 1 unit into `credit[i]`.
- Total credit equals `countOnes()`, proving that credit remains ≥ 0 throughout.

3. Examples & Trick Questions

Trick 1 (CLRS Exercise 16.2-1): Stack with Periodic Backup Copies

- **Problem:** A stack whose size never exceeds k has a backup copy of the entire stack made automatically after every k operations. Copying the stack takes time proportional to its size (at most k). Show that n operations cost $O(n)$ time using the accounting method.
 - **Analysis:**
 - An expensive copying operation costs at most k and occurs once every k operations.
 - To pay for this periodic cost, we levy an extra surcharge on each of the k intervening operations:
 - Standard PUSH: Actual cost = 1. Amortized cost = $2 + 1 = 3$.
 - \$1 pays for the push.
 - \$1 sits on the item to pay for its future POP.
 - \$1 is deposited into a global "backup fund".
 - Standard POP: Actual cost = 1. Amortized cost = $0 + 1 = 1$.
 - Actual pop is paid by the item's own \$1 credit.
 - \$1 is deposited into the global "backup fund".
 - In every block of k operations, exactly $k \times \$1 = \k is accumulated in the backup fund.
 - When the k -th operation triggers the copy (cost $\leq k$), the $\$k$ in the backup fund pays for it completely.
 - **Conclusion:** Total amortized cost per operation is at most $3 = O(1)$, so n operations take $O(n)$ actual time.
-

Trick 2 (CLRS Exercise 16.2-2): Operation Cost Dependent on Powers of 2

- **Problem:** Redo Exercise 16.1-3 using the accounting method: the i -th operation costs i if i is an exact power of 2, and 1 otherwise.
- **Analysis:**
 - Assign an amortized cost $\hat{c}_i = 3$ to every operation.
 - If i is not a power of 2:
 - Actual cost $c_i = 1$.
 - Credit deposited = $\hat{c}_i - c_i = 3 - 1 = \2 .
 - If $i = 2^j$ (a power of 2):
 - Actual cost $c_i = 2^j$.
 - Between 2^{j-1} and 2^j , there are 2^{j-1} operations.
 - The operations that are not powers of 2 in this range number $2^{j-1} - 1$.

- Each deposited \$2, accumulating $2(2^{j-1} - 1) = 2^j - 2$ credit.
 - Adding the \$3 charged for operation 2^j itself gives $(2^j - 2) + 3 = 2^j + 1$ available credit.
 - Actual cost is 2^j , which is fully covered with \$1 credit to spare.
 - **Conclusion:** Because credit never drops below zero and $\hat{c}_i = 3$, the amortized cost per operation is $O(1)$.
-

Trick 3 (CLRS Exercise 16.2-3): Counter with RESET Operation

- **Problem:** Implement a counter with INCREMENT and RESET (make all bits 0) such that any sequence of n operations takes $O(n)$ time on an initially zero counter. Bit reads/writes cost $\Theta(1)$.
 - **Analysis & Trap:**
 - *The Trap:* If RESET blindly sets all k bits to 0, it takes $\Theta(k)$ time. Alternating between INCREMENT and RESET would then take $\Theta(nk)$ time.
 - *The Solution:* Maintain a pointer or variable `high_order` that tracks the highest index containing a 1.
 - *Amortized Assignment:*
 - Set bit 0 $\rightarrow$ 1: Amortized cost = \$2 (\$1 to set, \$1 credit deposited on that bit).
 - RESET : Amortized cost = \$0.
 - Scan from `high_order` down to 0, resetting bits that are 1 to 0.
 - Each 1-bit holds \$1 of credit, which pays to examine and reset it.
 - Resetting `high_order` to 0 takes $O(1)$ overhead.
 - **Conclusion:** INCREMENT has amortized cost ≤ 2 , and RESET has amortized cost 0. Thus, n operations take $O(n)$ time.
-

4. Expected Questions & Model Answers

[2 Marks] Short-Answer Questions

Q1. In the accounting method, what happens when an operation's amortized cost exceeds its actual cost?

Answer:

The difference $(\hat{c}_i - c_i)$ is stored as **credit** on specific objects in the data structure. This credit is later consumed to pay for operations whose actual cost exceeds their amortized cost.

Q2. State the condition that must hold for total credit at any point in a sequence of n operations. Why is it required?

Answer:

The total credit must be nonnegative at all times:

$$\sum_{i=1}^n \hat{c}_i - \sum_{i=1}^n c_i \geq 0$$

If credit were negative, the total amortized cost would fall below the total actual cost, failing to serve as a valid upper bound.

Q3. What amortized costs are assigned to `PUSH`, `POP`, and `MULTIPOP` in the accounting method to achieve an $O(1)$ amortized bound?

Answer:

- `PUSH` : 2
 - `POP` : 0
 - `MULTIPOP` : 0
-

[5 Marks] Medium-Answer Questions

Q4. Using the accounting method, prove that the amortized cost per operation in a sequence of n `PUSH`, `POP`, and `MULTIPOP` operations on an initially empty stack is $O(1)$.

Answer:

1. Cost Assignments:

- `PUSH`: Assign amortized cost $\hat{c} = 2$.
- `POP`: Assign amortized cost $\hat{c} = 0$.
- `MULTIPOP`(S, k): Assign amortized cost $\hat{c} = 0$.

2. Credit Invariant:

- For every `PUSH`, actual cost is 1. The remaining \$1 is stored as credit directly on the pushed plate.
- For every `POP`, the actual cost of 1 is paid for by consuming the \$1 credit sitting on the plate being popped.
- For `MULTIPOP`(S, k), which pops $m = \min(|S|, k)$ plates, each of the m plates carries \$1 credit. These m credits pay for the actual cost m .

3. Nonnegativity & Bounding:

- Total credit in the stack at any time is $|S| \times \$1$. Since stack size $|S| \geq 0$, total credit is always nonnegative.
- Hence, total amortized cost bounds total actual cost:

$$\sum_{i=1}^n c_i \leq \sum_{i=1}^n \hat{c}_i \leq 2n = O(n)$$

- Amortized cost per operation is $O(1)$.

Q5. A stack has a maximum size of k . After every k operations, a full backup copy of the stack is made at an actual cost of k . Use the accounting method to show that n operations cost $O(n)$ time.

Answer:

1. Amortized Charges:

- Charge an extra surcharge of \$1 on each routine stack operation:
- $\hat{c}(\text{PUSH}) = 3$ (\$1 actual push, \$1 credit on item for pop, \$1 to backup fund).
- $\hat{c}(\text{POP}) = 1$ (\$0 for pop via item credit, \$1 to backup fund).

2. Credit Accumulation for Backup:

- The backup operation occurs strictly after every k operations.
- During any window of k operations, the backup fund accumulates:

$$k \times \$1 = \$k$$

3. Paying for Backup:

- Because stack size is at most k , copying the stack takes at most k units of time.
- The $\$k$ accumulated in the backup fund covers the backup cost entirely without reducing the fund below zero.

4. Conclusion:

- The total amortized cost for each operation is $O(1)$ (at most 3).
- Total actual cost for n operations is bounded by $\sum \hat{c}_i \leq 3n = O(n)$.

[10 Marks] Comprehensive Analytical Question

Q6. Provide a detailed amortized analysis of incrementing an initially zero k -bit binary counter using the accounting method.

1. Define the accounting model, assigning specific amortized costs to the bit operations.

2. Prove that the total credit in the counter is always nonnegative.
3. Derive the upper bound on the actual cost for a sequence of n INCREMENT operations.
4. Compare this approach with aggregate analysis for the same problem.

Answer:

1. Accounting Model & Cost Allocation

- Let 1 unit of cost (\$1) correspond to flipping a single bit ($0 \rightarrow 1$ or $1 \rightarrow 0$).
- **Bit Flip Charges:**
 - Flipping $0 \rightarrow 1$: Assign amortized cost $\hat{c} = 2$.
 - \$1 covers the actual work of setting the bit from 0 to 1.
 - \$1 is stored directly on that bit as credit.
- Flipping $1 \rightarrow 0$: Assign amortized cost $\hat{c} = 0$.
 - The actual work of resetting the bit is paid for by the \$1 credit already residing on that bit.
- **Analysis of INCREMENT Procedure:**
 - An increment scans low-order bits:
 - It flips m consecutive 1s to 0s inside the while loop. The amortized cost of these resets is $m \times \$0 = \0 .
 - If $i < k$, it flips at most one 0 to 1. The amortized cost is \$2.
 - Thus, for every call to INCREMENT, the total amortized cost is:

$$\hat{c}_{\text{INCREMENT}} \leq \$2$$

2. Proof of Credit Nonnegativity

- Let B_t denote the state of the counter after t operations, and let b_t be the number of 1-bits in B_t .
- Each 1-bit carries exactly \$1 of credit, and each 0-bit carries \$0 of credit.
- The total stored credit in the counter at step t is:

$$\text{Credit}_t = \sum_{i=0}^{k-1} \text{credit}[i] = b_t$$

- Because the counter contains a non-negative count of 1s ($0 \leq b_t \leq k$), we have:

$$\text{Credit}_t = b_t \geq 0 \quad \text{for all } t \geq 0$$

- The credit invariant $\sum_{i=1}^t \hat{c}_i \geq \sum_{i=1}^t c_i$ is preserved after every operation.

3. Derivation of Upper Bound on Actual Cost

- For a sequence of n operations starting from an initially zero counter:

$$\sum_{i=1}^n c_i \leq \sum_{i=1}^n \hat{c}_i$$

- Since each operation has an amortized cost $\hat{c}_i \leq 2$:

$$\sum_{i=1}^n \hat{c}_i \leq \sum_{i=1}^n 2 = 2n$$

- Therefore:

$$\sum_{i=1}^n c_i \leq 2n = O(n)$$

- The total actual cost is strictly $O(n)$, giving an amortized cost of $O(1)$ per `INCREMENT`.

4. Comparison: Accounting Method vs. Aggregate Analysis

- **Aggregate Analysis:**
 - Examines the system globally.
 - Calculates the exact flip frequencies of each individual bit position: bit i flips $\lfloor n/2^i \rfloor$ times.
 - Solves a geometric series $\sum_{i=0}^{k-1} \lfloor n/2^i \rfloor < 2n$ to find the total cost, then divides by n .
- **Accounting Method:**
 - Focuses locally on state transitions ($0 \rightarrow 1$ vs. $1 \rightarrow 0$).
 - Pre-pays the cost of resetting a bit at the moment it is set.
 - Bypasses geometric summations entirely by showing that each operation sets at most one bit to 1.
 - Establishes a concrete physical invariant: Credit = number of 1-bits.

Study Material: Section 8.1 – Polynomial-Time Reductions

1. Theory & In-Depth Explanation

1.1 The Concept of Polynomial-Time Reduction

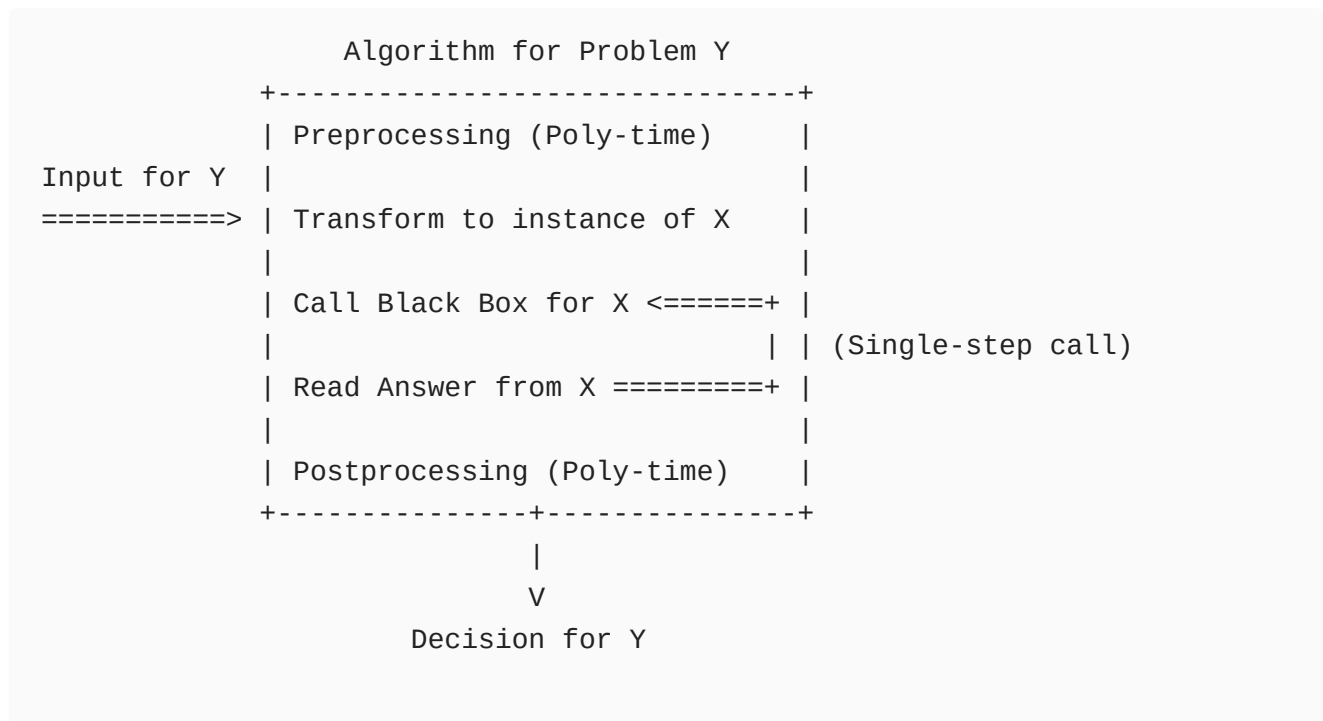
To formally characterize computationally hard problems, we compare the relative difficulty of problems using **reductions**.

Definition ($Y \leq_P X$):

We say problem Y is **polynomial-time reducible** to problem X (written $Y \leq_P X$, read " X is at least as hard as Y ") if arbitrary instances of problem Y can be solved using:

1. A polynomial number of standard computational steps, plus
2. A polynomial number of calls to a black box (oracle) that solves instances of problem X in a single step.

The model explicitly accounts for the time required to construct the input to the black box and read its output.



1.2 The Dual Role of Reductions

Reductions serve two opposite algorithmic purposes based on logical direction:

Fact (8.1) – Algorithm Design (Tractability)

If $Y \leq_P X$ and X can be solved in polynomial time, then Y can be solved in polynomial time.

- **Application:** Used to solve a new problem Y by converting it into an existing polynomial-time solvable problem X (e.g., $Bipartite\ Matching \leq_P Max-Flow$, $Image\ Segmentation \leq_P Min-Cut$).

Fact (8.2) – Hardness Proofs (Intractability / Contrapositive)

If $Y \leq_P X$ and Y cannot be solved in polynomial time, then X cannot be solved in polynomial time.

- **Application:** If problem Y is known to be intractable, proving $Y \leq_P X$ establishes that X must also be intractable. Hardness "spreads" from Y to X .

1.3 Decision Problems vs. Optimization Problems

To standardize reductions, problems are phrased as **decision problems** (yielding a binary YES / NO answer) rather than optimization problems:

- **Independent Set (Optimization):** Find an independent set of maximum size in graph G .
- **Independent Set (Decision):** Given graph G and an integer k , does G contain an independent set of size at least k ?

Equivalence via Binary Search

Solving the decision version allows solving the optimization version in polynomial time:

1. For graph G with n vertices, the maximum size lies in $[0, n]$.
2. Using binary search over $k \in [0, n]$, we query the decision black box $O(\log n)$ times.
3. The largest k that returns YES is the maximum independent set size.

1.4 Fundamental Graph Reductions: Independent Set $\equiv_P$ Vertex Cover

Definitions

- **Independent Set:** In $G = (V, E)$, a subset $S \subseteq V$ such that no two nodes in S are joined by an edge.
- **Decision Query:** Does G have an independent set of size $\geq k$?
- **Vertex Cover:** In $G = (V, E)$, a subset $S \subseteq V$ such that every edge $e \in E$ has at least one endpoint in S .
- **Decision Query:** Does G have a vertex cover of size $\leq k$?

Complement Duality Theorem: Fact (8.3)

Theorem: Let $G = (V, E)$ be a graph with $|V| = n$. A subset $S \subseteq V$ is an independent set if and only if its complement $V \setminus S$ is a vertex cover.

Graph $G = (V, E)$			
+-----+			
	$V \setminus S$	(Vertex Cover)	Every edge $e = (u, v)$ must have
		Size $\leq n - k$	at least one endpoint here.
+-----+			
	S	(Independent Set)	No edge can have both endpoints
		Size $\geq k$	in S (otherwise not independent).

+-----+

Proof:

1. ($\Rightarrow$) **Let S be an independent set.**

Consider an arbitrary edge $e = (u, v) \in E$. Because S is independent, u and v cannot both belong to S . Therefore, at least one of u or v must lie in $V \setminus S$. Thus, every edge has at least one endpoint in $V \setminus S$, making $V \setminus S$ a vertex cover.

2. ($\Leftarrow$) **Let $V \setminus S$ be a vertex cover.**

Suppose for contradiction that S is not an independent set. Then there exist nodes $u, v \in S$ joined by an edge $e = (u, v)$. Neither endpoint of e lies in $V \setminus S$, contradicting the fact that $V \setminus S$ covers all edges. Thus, no two nodes in S are joined by an edge, making S an independent set. ■

Bidirectional Reductions

- **Fact (8.4):** Independent Set $\leq_P$ Vertex Cover

To decide if G has an independent set of size $\geq k$, query the Vertex Cover black box on $(G, n - k)$.

- **Fact (8.5):** Vertex Cover $\leq_P$ Independent Set

To decide if G has a vertex cover of size $\leq k$, query the Independent Set black box on $(G, n - k)$.

1.5 Generalizing to Set Systems

Vertex Cover to Set Cover: Fact (8.6)

- **Set Cover Problem:** Given a universe U of elements, a family of subsets $\mathcal{S} = \{S_1, S_2, \dots, S_m\}$, and an integer k , does there exist a subcollection of at most k subsets whose union is U ?

Reduction Construction (Vertex Cover $\leq_P$ Set Cover):

- Given instance $(G = (V, E), k)$ of Vertex Cover:
 1. Define Universe $U = E$.
 2. For each vertex $v_i \in V$, construct subset $S_i = \{e \in E \mid e \text{ is incident to } v_i\}$.
 3. Keep the target parameter k unchanged.

Vertex Cover Instance G :	Set Cover Instance:
Vertices: $V = \{1, 2, \dots, n\}$	Subsets: $S_1, S_2, \dots, S_n$
Edges: $E = \{e_1, \dots, e_m\}$	Universe: $U = \{e_1, e_2, \dots, e_m\}$

Select $\leq k$ vertices

---> Select $\leq k$ subsets to cover U

Correctness:

- A set of vertices $\{v_{i_1}, \dots, v_{i_\ell}\}$ covers all edges in E if and only if the corresponding subsets $\{S_{i_1}, \dots, S_{i_\ell}\}$ cover all elements in universe U .

Independent Set to Set Packing: Fact (8.7)

- Set Packing Problem:** Given a universe U , a family of subsets $S_1, \dots, S_m \subseteq U$, and an integer k , does there exist a subcollection of at least k subsets that are pairwise disjoint?

Reduction Construction (Independent Set $\leq_P$ Set Packing):

- Given $(G = (V, E), k)$:
 - Define Universe $U = E$.
 - For each vertex $v \in V$, define $S_v = \{e \in E \mid e \text{ is incident to } v\}$.
 - Keep target size k .
- Correctness:** Subsets S_u and S_v are disjoint ($S_u \cap S_v = \emptyset$) if and only if vertices u and v share no common incident edge—i.e., edge $(u, v) \notin E$. Hence, selecting $\geq k$ disjoint subsets is equivalent to selecting $\geq k$ non-adjacent vertices.

2. Implementations & Walkthroughs

2.1 Reduction: Vertex Cover to Set Cover

Pseudocode

```
REDUCE-VC-TO-SC(G = (V, E), k)
1  U = E                                // Universe is the set of all
   edges
2  S = empty dictionary/map              // Maps vertex -> incident edge
   set
3  for each vertex v in V
4      S[v] = empty set
5  for each edge e = (u, v) in E
6      add e to S[u]
7      add e to S[v]
8  subsets_collection = values(S)
9  return (U, subsets_collection, k)    // Pass to Set Cover Black Box
```

C++ Implementation

```

#include <iostream>
#include <vector>
#include <set>
#include <utility>

// Graph Edge representation
struct Edge {
    int u, v;
    int id; // Unique edge identifier for Universe U

    bool operator<(const Edge& other) const {
        return id < other.id;
    }
};

// Graph representation
struct Graph {
    int numVertices;
    std::vector<Edge> edges;
};

// Set Cover Instance
struct SetCoverInstance {
    int universeSize; // |U| = |E|
    std::vector<std::set<int>> subsets; // S_i for each vertex
    int k; // Target bound
};

// Polynomial-Time Reduction Routine:  $O(|V| + |E|)$ 
SetCoverInstance reduceVertexCoverToSetCover(const Graph& G, int k) {
    SetCoverInstance sc;
    sc.universeSize = G.edges.size();
    sc.subsets.resize(G.numVertices);
    sc.k = k;

    // Populate subsets S_v with incident edge IDs
    for (const auto& edge : G.edges) {
        sc.subsets[edge.u].insert(edge.id);
        sc.subsets[edge.v].insert(edge.id);
    }

    return sc;
}

```

Walkthrough:

1. Universe U is indexed by edge IDs from 0 to $|E| - 1$.

2. For each vertex $v \in \{0, \dots, |V| - 1\}$, a set `subsets[v]` is initialized.
 3. Every edge $e = (u, v)$ is inserted into `subsets[u]` and `subsets[v]`, mapping incident edges to their respective endpoints.
 4. Total construction time is $O(|V| + |E|)$, which is strictly polynomial.
-

2.2 Optimization via Decision Oracle: Independent Set

Pseudocode

```
FIND-MAX-INDEPENDENT-SET-SIZE( $G = (V, E)$ )
1  low = 0
2  high = |V|
3  max_k = 0
4  while low <= high
5      mid = floor((low + high) / 2)
6      if INDEPENDENT-SET-ORACLE( $G, mid$ ) == TRUE
7          max_k = mid                      // Feasible, search higher
8          low = mid + 1
9      else
10         high = mid - 1                  // Infeasible, search lower
11 return max_k
```

C++ Implementation

```
#include <iostream>
#include <vector>
#include <functional>

// Generic binary search wrapper converting decision oracle to
// optimization solver
int solveMaxIndependentSetSize(
    int numVertices,
    const std::function<bool(int)>& decisionOracle)
{
    int low = 0;
    int high = numVertices;
    int optimalSize = 0;

    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (decisionOracle(mid)) {
            optimalSize = mid; // Candidate found; try to find a larger
one
            low = mid + 1;
        }
    }
    return optimalSize;
}
```

```

        } else {
            high = mid - 1;    // Size mid is infeasible; look lower
        }
    }
    return optimalSize;
}

```

Walkthrough:

* The function executes standard binary search over the range $[0, |V|]$.
 * Exactly $\lfloor \log_2(|V| + 1) \rfloor + 1$ oracle calls are made.
 * Demonstrates the polynomial equivalence between decision and optimization variants.

3. Examples & Trick Questions

Trick 1: Direction of Reduction Trap

- **Scenario:** A student wants to prove that problem B is computationally hard. They know problem A is hard, and they construct a polynomial-time reduction showing $B \leq_P A$.
- **Pitfall:** This proves that B is *at most as hard as* A . If A is hard, B could still be trivial (e.g., solvable in $O(n)$ time).
- **Rule:** To prove B is hard using a known hard problem A , you must reduce **from** the known hard problem **to** the target problem:

$$A \leq_P B$$

Trick 2: Complement Graph vs. Complement Vertex Set

- **Scenario:** Confusing the reduction between *Independent Set* and *Clique* with the reduction between *Independent Set* and *Vertex Cover*.
- **Clarification:**
 - **Independent Set** $\leftrightarrow$ **Vertex Cover:** Operates on the **same graph** G . S is independent in $G \iff V \setminus S$ is a vertex cover in G . Target size changes: $k \rightarrow n - k$.
 - **Independent Set** $\leftrightarrow$ **Clique:** Operates on the **complement graph** $\overline{G} = (V, \overline{E})$. S is independent in $G \iff S$ forms a clique in $\overline{G}$. Target size stays k .

Trick 3: Handling Isolated Vertices in $\text{Vertex Cover} \leq_P \text{Set Cover}$

- **Question:** In the reduction $\text{Vertex Cover} \leq_P \text{Set Cover}$, what happens if graph G contains isolated vertices (vertices with degree 0)?
 - **Analysis:**
 - For an isolated vertex w , its incident edge set is empty: $S_w = \emptyset$.
 - In Set Cover, choosing an empty set covers 0 elements of $U = E$.
 - A minimal Set Cover will never select S_w unless all elements are already covered and extra sets are allowed.
 - In Vertex Cover, an isolated vertex never helps cover any edge. A minimum vertex cover will never include w .
 - **Conclusion:** The equivalence holds without modifications.
-

4. Expected Questions & Model Answers

[2 Marks] Short-Answer Questions

Q1. Define what it means to state $Y \leq_P X$.

Answer:

$Y \leq_P X$ denotes that problem Y is polynomial-time reducible to problem X . This means arbitrary instances of Y can be solved using a polynomial number of standard computational steps plus a polynomial number of calls to an oracle that solves instances of X in unit time.

Q2. State the contrapositive formulation of polynomial-time reducibility and explain its significance.

Answer:

If $Y \leq_P X$ and Y cannot be solved in polynomial time, then X cannot be solved in polynomial time. Its significance is that it enables proving a new problem X is computationally intractable by reducing a known intractable problem Y to X .

Q3. Given an oracle for the decision version of Vertex Cover, how many oracle calls are needed to determine the minimum vertex cover size of an n -node graph?

Answer:

$O(\log n)$ calls. Since the minimum cover size lies within $[0, n]$, binary search over candidate sizes k finds the minimum feasible k in at most $\lceil \log_2(n+1) \rceil$ calls.

[5 Marks] Medium-Answer Questions

Q4. Prove that in any graph $G = (V, E)$, a subset $S \subseteq V$ is an independent set if and only if $V \setminus S$ is a vertex cover.

Answer:

Let $G = (V, E)$ be an arbitrary undirected graph.

1. Forward Direction ($\Rightarrow$):

Assume S is an independent set. Consider any edge $e = (u, v) \in E$. Since S is independent, u and v cannot both belong to S (otherwise e connects two nodes in S). Therefore, at least one endpoint (u or v) must lie outside S , meaning it belongs to $V \setminus S$. Because every edge in E has at least one endpoint in $V \setminus S$, $V \setminus S$ is a vertex cover.

2. Reverse Direction ($\Leftarrow$):

Assume $V \setminus S$ is a vertex cover. Suppose for contradiction that S is not an independent set. Then there must exist two nodes $u, v \in S$ such that $e = (u, v) \in E$. Since neither u nor v belongs to $V \setminus S$, the edge e has neither endpoint in $V \setminus S$, contradicting the premise that $V \setminus S$ is a vertex cover. Hence, no two nodes in S share an edge, proving S is an independent set.

Thus, S is independent if and only if $V \setminus S$ is a vertex cover.

Q5. Show that Independent Set $\leq_P$ Set Packing.

Answer:

To show Independent Set $\leq_P$ Set Packing, we transform an arbitrary instance of Independent Set ($G = (V, E), k$) into an instance of Set Packing ($(U, \mathcal{S}, k)$):

1. Construction:

- Universe $U = E$ (the set of all edges in G).
- For each vertex $v \in V$, construct a set $S_v = \{e \in E \mid e \text{ is incident to } v\}$.
- Set family $\mathcal{S} = \{S_v \mid v \in V\}$.
- Set target packing size to k .
- This transformation takes $O(|V| + |E|)$ time, which is polynomial.

2. Correctness Argument:

- Two sets $S_u, S_v \in \mathcal{S}$ are disjoint ($S_u \cap S_v = \emptyset$) if and only if vertices u and v do not share an edge in G , which means $(u, v) \notin E$.
- Consequently, a collection of k subsets $\{S_{v_1}, \dots, S_{v_k}\}$ is pairwise disjoint if and only if the corresponding vertices $\{v_1, \dots, v_k\}$ share no edges, forming an independent set of size k in G .

- The answer to the Set Packing instance is YES if and only if the answer to the Independent Set instance is YES.

[10 Marks] Comprehensive Analytical Question

Q6. Polynomial-Time Reduction from Vertex Cover to Set Cover.

1. Define both the Vertex Cover problem and the Set Cover problem in their decision forms.
2. Describe the polynomial-time reduction algorithm from Vertex Cover to Set Cover.
3. Prove both directions of correctness for the reduction.
4. Given a graph with $V = \{1, 2, 3, 4\}$ and $E = \{(1, 2), (2, 3), (3, 4), (1, 4), (2, 4)\}$, construct the corresponding Set Cover instance for $k = 2$.

Answer:

1. Problem Definitions

- **Vertex Cover (Decision):** Given an undirected graph $G = (V, E)$ and an integer $k \leq |V|$, does there exist a subset $S \subseteq V$ of size $|S| \leq k$ such that for every edge $(u, v) \in E$, either $u \in S$ or $v \in S$?
- **Set Cover (Decision):** Given a finite universe U , a collection of subsets $\mathcal{S} = \{S_1, S_2, \dots, S_m\}$ where $S_i \subseteq U$, and an integer $k \leq m$, does there exist a subcollection $\mathcal{C} \subseteq \mathcal{S}$ of size $|\mathcal{C}| \leq k$ such that $\bigcup_{S_i \in \mathcal{C}} S_i = U$?

2. Reduction Algorithm

Given an arbitrary instance $(G = (V, E), k)$ of Vertex Cover:

1. Define the universe $U = E$.
2. For each vertex $v \in V$, construct a set S_v containing all edges incident to v :

$$S_v = \{e \in E \mid v \text{ is an endpoint of } e\}$$

3. Let $\mathcal{S} = \{S_v \mid v \in V\}$.
4. Output the Set Cover instance $(U, \mathcal{S}, k)$.

- **Complexity:** Constructing U and the subsets takes $O(|V| + |E|)$ time, which is polynomial in the input size.

3. Proof of Correctness

We prove that G has a vertex cover of size $\leq k$ if and only if $(U, \mathcal{S})$ has a set cover of size $\leq k$.

- **Direction 1 ($\Rightarrow$):**

Suppose G has a vertex cover $V' \subseteq V$ with $|V'| \leq k$.

Consider the subcollection of sets $\mathcal{C} = \{S_v \mid v \in V'\}$. The number of sets is $|\mathcal{C}| = |V'| \leq k$.

For any element $e = (u, v) \in U$, because V' is a vertex cover, at least one of u or v must be in V' . If $u \in V'$, then $e \in S_u \subseteq \bigcup_{S \in \mathcal{C}} S$. Thus, every element of U is covered, meaning $\mathcal{C}$ is a valid Set Cover of size $\leq k$.

- **Direction 2 ($\Leftarrow$):**

Suppose there exists a subcollection $\mathcal{C} \subseteq \mathcal{S}$ of size $|\mathcal{C}| \leq k$ that covers U .

Construct the vertex subset $V' = \{v \in V \mid S_v \in \mathcal{C}\}$. Clearly, $|V'| = |\mathcal{C}| \leq k$.

Consider any edge $e = (u, v) \in E$. Since $e \in U$ and $\mathcal{C}$ covers U , e must belong to some set $S_w \in \mathcal{C}$. By definition of our sets, $e \in S_w$ implies that w is an endpoint of e , so either $w = u$ or $w = v$. Therefore, at least one of u or v belongs to V' , confirming that V' covers edge e . Since this holds for all edges, V' is a valid Vertex Cover of size $\leq k$.

4. Instance Construction Example

Given:

- $V = \{1, 2, 3, 4\}$
- $E = \{e_1 = (1, 2), e_2 = (2, 3), e_3 = (3, 4), e_4 = (1, 4), e_5 = (2, 4)\}$
- $k = 2$

Constructed Set Cover Instance:

- **Universe:**

$$U = \{e_1, e_2, e_3, e_4, e_5\}$$

- **Subsets:**

- $S_1 = \{e_1, e_4\}$ (incident to vertex 1)
- $S_2 = \{e_1, e_2, e_5\}$ (incident to vertex 2)
- $S_3 = \{e_2, e_3\}$ (incident to vertex 3)
- $S_4 = \{e_3, e_4, e_5\}$ (incident to vertex 4)

- **Target:** $k = 2$

- **Verification:** Selecting vertices $\{2, 4\}$ forms a vertex cover of size 2. Correspondingly, choosing subsets $\{S_2, S_4\}$ gives:

$$S_2 \cup S_4 = \{e_1, e_2, e_5\} \cup \{e_3, e_4, e_5\} = \{e_1, e_2, e_3, e_4, e_5\} = U$$

This yields a valid set cover of size 2.

Study Material: Sections 8.3 & 8.4 – Efficient Certification, the Class NP, and NP-Completeness

1. Theory & In-Depth Explanation

1.1 The Duality of Solving vs. Certifying

A central theme of computational complexity is the asymmetry between **finding a solution** from scratch and **verifying (certifying) a proposed solution**:

- **Finding a solution**: Searching through an exponentially large configuration space (e.g., finding an Independent Set of size $\geq k$ or finding a satisfying assignment for 3-SAT).
- **Verifying a solution**: Given candidate evidence (a *certificate* or *witness*), checking whether it satisfies the problem constraints. Verification is often achievable in polynomial time.

```
FINDING A SOLUTION (Intrinsically Hard)
Input s -----> [ ??? Algorithmic Search ??? ] ----->
"YES" / "NO"
                                (Explore exponentially many candidates)

VERIFYING A SOLUTION (Efficient Certification)
Input s      ----\
               +--> [ Efficient Certifier B(s, t) ] ----->
"YES" / "NO"
    Certificate t ----/      (Deterministic Polynomial-Time
Verification)
```

1.2 Formal Definitions: Decision Problems, P, and NP

1. Formal Language Framework

- Instances of a problem are encoded as finite binary strings $s \in \{0, 1\}^*$ of length $|s|$.
- A **decision problem** X is identified with the set of strings for which the answer is "YES":

$$X \subseteq \{0, 1\}^*$$

2. The Class P (Polynomial Time)

$P = \{X \mid \exists \text{ algorithm } A \text{ and polynomial } p \text{ such that } A(s) \text{ runs in } O(p(|s|)) \text{ and } A(s) = \text{YES} \iff s \in X\}$

3. Efficient Certifier and the Class NP (Nondeterministic Polynomial Time)

An algorithm $B(s, t)$ is an **efficient certifier** for problem X if:

1. B is a deterministic polynomial-time algorithm taking two input arguments: an instance string s and a certificate string t .
2. There exists a polynomial $p(\cdot)$ such that for every string s :

$$s \in X \iff \exists t \text{ with } |t| \leq p(|s|) \text{ such that } B(s, t) = \text{YES}$$

$$\mathbf{NP} = \{X \mid X \text{ has an efficient certifier}\}$$

4. The Relation $\mathbf{P} \subseteq \mathbf{NP}$ (Fact 8.10)

- **Proof:** Let $X \in \mathbf{P}$. There exists a deterministic polynomial-time algorithm $A(s)$ that decides X . We construct an efficient certifier $B(s, t)$ that simply ignores the certificate t and evaluates $A(s)$. Because A runs in polynomial time, B runs in polynomial time, and $B(s, t) = \text{YES} \iff s \in X$. Thus, $\mathbf{P} \subseteq \mathbf{NP}$.

1.3 Examples of Certificates for Problems in NP

Problem	Instance s	Certificate t	Certifier Check $B(s, t)$ in Polynomial Time
3-SAT	Boolean formula in 3-CNF	Assignment of truth values to all n variables	Evaluate each clause; verify all clauses evaluate to TRUE .
Independent Set	Graph $G = (V, E)$ and integer k	Subset of vertices $S \subseteq V$	Check that $ S \geq k$, and for all pairs $u, v \in S$, $(u, v) \notin E$.
Vertex Cover	Graph $G = (V, E)$ and integer k	Subset of vertices $S \subseteq V$	Check that $ S \leq k$, and for every edge $e \in E$, at least one endpoint is in S .
Set Cover	Universe U , subsets $S_1, \dots, S_m$, integer k	Collection of indices $\{i_1, \dots, i_\ell\}$	Check that $\ell \leq k$ and $\bigcup_{j=1}^{\ell} S_{i_j} = U$.

1.4 The Concept of NP-Completeness

Definition

A problem X is **NP-complete** if:

1. $X \in \mathbf{NP}$ (Membership).
2. For every problem $Y \in \mathbf{NP}$, $Y \leq_P X$ (Hardness).

Foundational Property: Fact (8.12)

If any **NP**-complete problem X can be solved in polynomial time, then $\mathbf{P} = \mathbf{NP}$.
Conversely, if $\mathbf{P} \neq \mathbf{NP}$, no **NP**-complete problem can be solved in polynomial time.

1.5 The Cook-Levin Theorem & Circuit Satisfiability (Fact 8.13)

To establish the first **NP**-complete problem without reducing from another known **NP**-complete problem, Cook and Levin proved that **Circuit Satisfiability (Circuit-SAT)** can simulate any arbitrary problem in **NP**.

Combinational Boolean Circuits

A circuit K is a labeled directed acyclic graph (DAG):

- **Sources** (in-degree 0): Labeled with fixed constants (0 or 1) or variable **inputs** $x_1, \dots, x_m$.
- **Gates**: Labeled with $\neg$ (in-degree 1), $\vee$ (in-degree 2), or $\wedge$ (in-degree 2).
- **Output Node** (out-degree 0): Emits the final computed Boolean evaluation.
- **Circuit-SAT Problem**: Given circuit K , does there exist an input variable assignment that makes the output 1?

```

Sources/Inputs:  [ 1 ]      [ 0 ]      [ x_1 ]      [ x_2 ]      [ x_3 ]
                  \        /          \        /          /
Row 1 Gates:      [ AND ]                [ OR ]                [ OR ]
                  \        /          /        \        /
Row 2 Gates:      [      NOT      ]      [      AND      ]
                  \        /          /
Output Gate:      [      AND      ] ==> Evaluates to 1?

```

Core Idea of the Proof ($X \leq_P \text{Circuit-SAT}$ for any $X \in \mathbf{NP}$)

1. For any problem $X \in \mathbf{NP}$, there exists an efficient certifier $B(s, t)$ running in time $O(p(|s|))$.
2. Any algorithm executing on a computer for T steps on an input of size N can be simulated by a Boolean combinational circuit of size $O(T^2)$ (or $O(T \log T)$).
3. For a specific instance s of length n :
 - Build circuit K simulating $B(s, t)$.
 - Hard-code the n bits of instance s as **constant sources**.

- Leave the $p(n)$ bits of certificate t as **free input sources**.

4. The circuit K is satisfiable $\iff \exists t$ such that $B(s, t) = \text{YES} \iff s \in X$.

1.6 Proving Further Problems NP-Complete

Transitivity Rule: Fact (8.14)

If Y is NP-complete, $X \in \text{NP}$, and $Y \leq_P X$, then X is NP-complete.

The Reduction: Circuit-SAT $\leq_P$ 3-SAT (Fact 8.15)

To reduce an arbitrary circuit K to 3-SAT:

1. **Assign Variables:** Assign a Boolean variable x_v to every node (source and gate) v in K .
2. **Encode Gate Consistency:**

- **NOT Gate** ($v = \neg u \implies x_v \leftrightarrow \neg x_u$):

$$(x_v \vee x_u) \wedge (\overline{x_v} \vee \overline{x_u})$$

- **OR Gate** ($v = u \vee w \implies x_v \leftrightarrow (x_u \vee x_w)$):

$$(\overline{x_v} \vee x_u \vee x_w) \wedge (x_v \vee \overline{x_u}) \wedge (x_v \vee \overline{x_w})$$

- **AND Gate** ($v = u \wedge w \implies x_v \leftrightarrow (x_u \wedge x_w)$):

$$(x_v \vee \overline{x_u} \vee \overline{x_w}) \wedge (\overline{x_v} \vee x_u) \wedge (\overline{x_v} \vee x_w)$$

3. Hard-Code Constants & Output:

- For constant source $v = 1$: Add clause (x_v) .
- For constant source $v = 0$: Add clause $(\overline{x_v})$.
- For output gate o : Add clause (x_o) (forces circuit output to evaluate to 1).

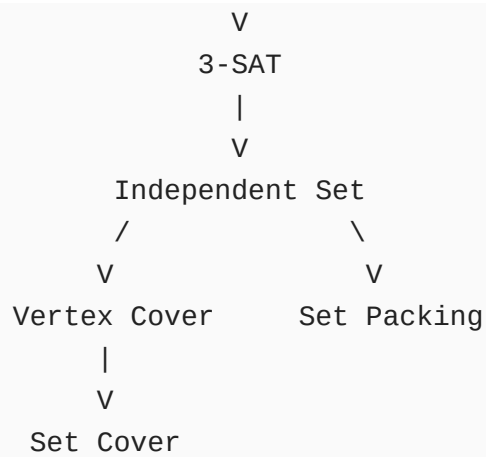
4. Pad to Exactly 3 Literals per Clause:

- Introduce 4 auxiliary variables z_1, z_2, z_3, z_4 .
- Enforce $z_1 = 0$ and $z_2 = 0$ using all 4 combinations of z_3, z_4 :

$$(z_i \vee z_3 \vee z_4) \wedge (z_i \vee z_3 \vee \overline{z_4}) \wedge (z_i \vee \overline{z_3} \vee z_4) \wedge (z_i \vee \overline{z_3} \vee \overline{z_4}) \quad \text{for } i \in \{1, 2\}$$

- Transform 1-literal clause (t) into: $(t \vee z_1 \vee z_2)$.
- Transform 2-literal clause $(t_1 \vee t_2)$ into: $(t_1 \vee t_2 \vee z_1)$.
- Clauses with 3 literals remain unchanged.

The resulting 3-CNF formula is satisfiable $\iff$ the original circuit K is satisfiable.



2. Implementations & Walkthroughs

2.1 Independent Set Certifier

Pseudocode

```
CERTIFIER-INDEPENDENT-SET( $G = (V, E)$ ,  $k$ ,  $S$ )
1  // Step 1: Check certificate size
2  if  $|S| < k$ 
3      return NO
4
5  // Step 2: Ensure all certified nodes exist in  $V$ 
6  for each node  $u$  in  $S$ 
7      if  $u$  not in  $V$ 
8          return NO
9
10 // Step 3: Check independence condition
11 for each pair of distinct nodes  $u, v$  in  $S$ 
12     if  $(u, v) \in E$ 
13         return NO
14
15 return YES
```

C++ Implementation

```
#include <iostream>
#include <vector>
#include <unordered_set>

struct Graph {
    int numVertices;
```

```

    std::vector<std::vector<int>> adjMatrix; // 1 if edge exists, 0
    otherwise
};

// Deterministic Polynomial-Time Certifier B(s, t)
// s = (G, k), certificate t = proposedSet
bool verifyIndependentSet(const Graph& G, int k, const std::vector<int>&
proposedSet) {
    // 1. Verify certificate size constraint: |S| >= k
    if (static_cast<int>(proposedSet.size()) < k) {
        return false;
    }

    // 2. Validate vertex bounds
    for (int v : proposedSet) {
        if (v < 0 || v >= G.numVertices) {
            return false;
        }
    }

    // 3. Verify no edge connects any two vertices in proposedSet:
    O(|S|^2)
    for (size_t i = 0; i < proposedSet.size(); ++i) {
        for (size_t j = i + 1; j < proposedSet.size(); ++j) {
            int u = proposedSet[i];
            int v = proposedSet[j];
            if (G.adjMatrix[u][v] == 1) {
                return false; // Edge found between members; invalid
certificate
            }
        }
    }

    return true; // Valid independent set certificate
}

```

Walkthrough:

1. Verifies that the certificate contains at least k vertices in $O(1)$ operations.
2. Performs an $O(|S|^2)$ pairwise check against the adjacency matrix. Since $|S| \leq |V|$, total verification runs in $O(|V|^2)$ deterministic polynomial time.

2.2 Gate Transformation & Clause Padding (Circuit-SAT $\rightarrow$ 3-SAT)

Pseudocode

```

CONVERT-GATE-TO-3SAT(gate_type, v, u, w, z1, z2)
1  clauses = empty list
2  if gate_type == NOT
3      // Raw clauses: (xv or xu) and (not xv or not xu)
4      clauses.append(MAKE-3-LITERAL(xv, xu, z1))
5      clauses.append(MAKE-3-LITERAL(not xv, not xu, z1))
6  else if gate_type == OR
7      // Raw clauses: (not xv or xu or xw), (xv or not xu), (xv or not
xw)
8      clauses.append((not xv, xu, xw))
9      clauses.append(MAKE-3-LITERAL(xv, not xu, z1))
10     clauses.append(MAKE-3-LITERAL(xv, not xw, z1))
11 else if gate_type == AND
12     // Raw clauses: (xv or not xu or not xw), (not xv or xu), (not xv
or xw)
13     clauses.append((xv, not xu, not xw))
14     clauses.append(MAKE-3-LITERAL(not xv, xu, z1))
15     clauses.append(MAKE-3-LITERAL(not xv, xw, z1))
16 return clauses

```

C++ Implementation

```

#include <iostream>
#include <vector>
#include <string>

// Representation of a Literal: positive integer for variable, negative
for negation
struct Literal {
    int var; // Variable ID: 1, 2, 3...
    bool isNegated;

    std::string toString() const {
        return (isNegated ? "~x" : "x") + std::to_string(var);
    }
};

struct Clause3 {
    Literal l1, l2, l3;
};

// Generates the 8 fixed clauses that force z1 = 0 and z2 = 0
std::vector<Clause3> createZeroEnforcementClauses(int z1, int z2, int z3,
int z4) {
    std::vector<Clause3> clauses;
    int targets[2] = {z1, z2};

```

```

    for (int zi : targets) {
        clauses.push_back({{zi, false}, {z3, false}, {z4, false}});
        clauses.push_back({{zi, false}, {z3, false}, {z4, true}});
        clauses.push_back({{zi, false}, {z3, true}, {z4, false}});
        clauses.push_back({{zi, false}, {z3, true}, {z4, true}});
    }
    return clauses;
}

// Converts a 2-literal clause (l1 or l2) into a 3-literal clause (l1 or
// l2 or z1)
Clause3 pad2To3(Literal l1, Literal l2, int z1) {
    return {l1, l2, {z1, false}};
}

// Converts a 1-literal clause (l1) into a 3-literal clause (l1 or z1 or
// z2)
Clause3 pad1To3(Literal l1, int z1, int z2) {
    return {l1, {z1, false}, {z2, false}};
}

```

Walkthrough:

1. The auxiliary variables z_1, z_2 are constrained to evaluate strictly to 0 (FALSE) across all truth assignments by pairing them with all four combinations of z_3, z_4 .
2. Any single-literal or double-literal intermediate clause is padded with z_1 and z_2 , guaranteeing exact clause lengths of 3 while preserving the truth value of the original clause.

3. Examples & Trick Questions

Trick 1: The Asymmetry of Certification (NP vs. co-NP)

- **Question:** If 3-SAT is in NP, does that automatically mean deciding whether a 3-SAT formula is **unsatisfiable** is also in NP?
- **Analysis:**
 - For a satisfiable formula, a single n -bit assignment serves as a succinct certificate.
 - For an unsatisfiable formula, what short certificate could prove that *none* of the 2^n possible assignments satisfy the formula? Exhibiting one failing assignment does not prove all others fail.
- **Conclusion:** No. Proving unsatisfiability belongs to the complement class co-NP. Whether $\text{NP} = \text{co-NP}$ remains an open problem.

Trick 2: Pitfall of Literal Duplication During Clause Padding

- **Question:** When reducing SAT to 3-SAT, why not simply convert a 2-literal clause $(x_1 \vee x_2)$ to $(x_1 \vee x_2 \vee x_2)$ and a 1-literal clause (x_1) to $(x_1 \vee x_1 \vee x_1)$ instead of using dummy variables?
 - **Analysis:**
 - In strict formal definitions of 3-SAT, clauses are defined as sets of three **distinct** literals.
 - In many systems, clauses with repeated literals $(x_1 \vee x_1 \vee x_1)$ are treated as degenerate 1-clauses, which can fail downstream reductions that rely on gadget structures (such as the reduction from 3-SAT to Independent Set, which constructs a triangle gadget for each clause).
 - **Conclusion:** Introducing distinct auxiliary variables constrained to `FALSE` ensures strict conformance to the standard 3-literal format.
-

Trick 3: The Common Reducibility Direction Mistake

- **Question:** To prove that problem Q is **NP**-complete, a student proves that $Q \leq_P \text{3-SAT}$. Is this proof correct?
- **Analysis:**
 - Proving $Q \leq_P \text{3-SAT}$ shows that Q is no harder than 3-SAT. This only proves that $Q \in \text{NP}$ (since 3-SAT is in **NP**).
 - To prove Q is **NP**-complete, one must prove that a known **NP**-complete problem reduces to Q :

$$\text{3-SAT} \leq_P Q$$

- **Conclusion:** The student inverted the reduction direction.
-

4. Expected Questions & Model Answers

[2 Marks] Short-Answer Questions

Q1. Define the complexity class NP.

Answer:

NP (Nondeterministic Polynomial Time) is the class of decision problems X for which there exists an efficient certifier algorithm $B(s, t)$ running in polynomial time such that an instance $s \in X$ if and only if there exists a certificate string t with length bounded by a polynomial in $|s|$ where $B(s, t) = \text{YES}$.

Q2. State the two requirements necessary to prove that a problem X is NP-complete.

Answer:

1. $X \in \mathbf{NP}$ (membership in $\mathbf{NP}$ via an efficient certifier).
2. For some known $\mathbf{NP}$ -complete problem Y , $Y \leq_P X$ (polynomial-time hardness).

Q3. What is the significance of the Cook-Levin Theorem?

Answer:

The Cook-Levin Theorem proved that Circuit Satisfiability is $\mathbf{NP}$ -complete, establishing the existence of a first $\mathbf{NP}$ -complete problem by showing that every problem in $\mathbf{NP}$ can be reduced to it in polynomial time.

[5 Marks] Medium-Answer Questions

Q4. Prove that $\mathbf{P} \subseteq \mathbf{NP}$.

Answer:

1. **Premise:** Let X be an arbitrary problem in $\mathbf{P}$. By definition of $\mathbf{P}$, there exists an algorithm $A(s)$ that decides whether $s \in X$ in deterministic time bounded by a polynomial $p(|s|)$.
 2. **Certifier Construction:** We construct a candidate certifier algorithm $B(s, t)$ for problem X . When given instance s and proposed certificate t , $B(s, t)$ ignores t and directly runs $A(s)$, returning the output of $A(s)$.
 3. **Polynomial Time Verification:** Since $A(s)$ runs in $O(p(|s|))$ time, $B(s, t)$ also runs in deterministic polynomial time in $|s|$.
 4. **Correctness:**
 - If $s \in X$, then $A(s) = \text{YES}$, so $B(s, t) = \text{YES}$ for every certificate t .
 - If $s \notin X$, then $A(s) = \text{NO}$, so $B(s, t) = \text{NO}$ for every certificate t .
 5. **Conclusion:** B is a valid efficient certifier for X . Since this holds for every $X \in \mathbf{P}$, it follows that $\mathbf{P} \subseteq \mathbf{NP}$.
-

Q5. Describe the padding technique used to convert clauses with fewer than three literals into equivalent 3-literal clauses.

Answer:

To transform an arbitrary CNF formula with clause lengths ≤ 3 into an exact 3-CNF formula:

1. **Auxiliary Variables:** Introduce four new Boolean variables z_1, z_2, z_3, z_4 .
2. **Forcing Zeros:** Force $z_1 = 0$ and $z_2 = 0$ in all satisfying assignments by adding all 4 combinations of z_3, z_4 for each $z_i \in \{z_1, z_2\}$:

$$(z_i \vee z_3 \vee z_4) \wedge (z_i \vee z_3 \vee \overline{z_4}) \wedge (z_i \vee \overline{z_3} \vee z_4) \wedge (z_i \vee \overline{z_3} \vee \overline{z_4})$$

These clauses can only be satisfied if $z_1 = 0$ and $z_2 = 0$.

3. Clause Replacement:

- Any 1-literal clause (t) is replaced by $(t \vee z_1 \vee z_2)$.
 - Any 2-literal clause $(t_1 \vee t_2)$ is replaced by $(t_1 \vee t_2 \vee z_1)$.
 - Any 3-literal clause $(t_1 \vee t_2 \vee t_3)$ remains unchanged.
4. **Equivalence:** Because $z_1 = z_2 = 0$, the padded clause evaluates to **TRUE** if and only if the original literal(s) evaluate to **TRUE**.

[10 Marks] Comprehensive Analytical Question

Q6. The Reduction from Circuit Satisfiability to 3-SAT.

1. Define the Circuit Satisfiability problem and the 3-SAT problem.
2. Provide the step-by-step polynomial-time reduction algorithm that maps any circuit K to a 3-CNF formula Φ .
3. Prove that circuit K is satisfiable if and only if formula Φ is satisfiable.
4. Analyze the size and construction time of Φ in terms of the number of gates and wires in K .

Answer:

1. Problem Definitions

- **Circuit Satisfiability (Circuit-SAT):** Given a combinational Boolean circuit K composed of directed acyclic wires and gates ($\neg, \vee, \wedge$) with variable inputs and a single output node, decide if there exists an assignment of truth values to the input variables such that the circuit output evaluates to 1.
- **3-Satisfiability (3-SAT):** Given a Boolean formula Φ in Conjunctive Normal Form (CNF) where each clause contains exactly three distinct literals, decide if there exists a truth assignment to its variables that satisfies Φ .

2. Reduction Algorithm

Given circuit K with vertices V (sources and gates):

1. **Variables:** Define a Boolean variable x_v for every node $v \in V$.
2. **Gate Consistency Clauses:**

- If v is a $\neg$ gate with input wire from u : Add $(x_v \vee x_u) \wedge (\overline{x_v} \vee \overline{x_u})$.
- If v is an $\vee$ gate with input wires from u, w : Add $(\overline{x_v} \vee x_u \vee x_w) \wedge (x_v \vee \overline{x_u}) \wedge (x_v \vee \overline{x_w})$.
- If v is an $\wedge$ gate with input wires from u, w : Add $(x_v \vee \overline{x_u} \vee \overline{x_w}) \wedge (\overline{x_v} \vee x_u) \wedge (\overline{x_v} \vee x_w)$.

3. Sources and Output Constraints:

- For fixed constant source $v = 1$: Add clause (x_v) .
- For fixed constant source $v = 0$: Add clause $(\overline{x_v})$.
- For output node o : Add clause (x_o) .

4. Padding to 3 Literals:

- Add auxiliary variables z_1, z_2, z_3, z_4 .
- Add 8 clauses forcing $z_1 = 0$ and $z_2 = 0$.
- Pad 1-literal clauses (t) into $(t \vee z_1 \vee z_2)$.
- Pad 2-literal clauses $(t_1 \vee t_2)$ into $(t_1 \vee t_2 \vee z_1)$.

3. Proof of Equivalence (K is satisfiable $\iff \Phi$ is satisfiable)

• ($\Rightarrow$) Let K be satisfiable:

There exists an assignment to the circuit input variables causing output node o to evaluate to 1. Evaluate the circuit top-to-bottom, assigning to each variable x_v the Boolean value computed at gate v . Set $z_1 = z_2 = 0$ (and assign arbitrary values to z_3, z_4 that satisfy their enforcement clauses). Because every gate computes its Boolean operation faithfully, all gate clauses are satisfied. Since the output evaluated to 1, clause (x_o) is satisfied. Thus, Φ is satisfied.

• ($\Leftarrow$) Let Φ be satisfiable:

There exists a truth assignment to all variables $\{x_v\}$ and $\{z_i\}$ satisfying Φ . The clauses enforce $z_1 = z_2 = 0$. For each gate v , the gate clauses enforce $x_v = \text{operation}(x_{\text{inputs}})$. Thus, the values assigned to x_v correspond to the values computed by the circuit on the inputs. Finally, because the clause $(x_o \vee z_1 \vee z_2)$ is satisfied and $z_1 = z_2 = 0$, x_o must be 1. Therefore, the circuit output evaluates to 1, making K satisfiable.

4. Complexity Analysis

- Let K have $|V|$ nodes and $|E|$ wires.
- Number of variables in Φ : $|V| + 4 = O(|V|)$.
- Number of clauses in Φ :
 - Each gate produces at most 3 clauses.
 - Constant sources and output produce 1 clause each.
 - Forcing clauses contribute 8 clauses.
 - Total clauses $\leq 3|V| + 9 = O(|V|)$.
- Each clause contains exactly 3 literals.

- Generating variables and writing clauses takes $O(|V| + |E|)$ time, which is linear in the size of the circuit and strictly polynomial.